

Adaptive Boosting (AdaBoost)

From Decision Stumps to a Strong Classifier

Sourav Karmakar

Outline

- 1 Introduction to AdaBoost
- 2 Decision Stumps: The Weak Learners
- 3 Example Dataset
- 4 AdaBoost Step-by-Step
- 5 Algorithm Summary
- 6 Why Does AdaBoost Work?
- 7 Practical Considerations
- 8 Applications & Limitations
- 9 Final Review

Introduction to AdaBoost

The Three Main Ideas Behind AdaBoost

1. Weak Learners

AdaBoost combines many **weak learners** (stumps) to make classifications.

2. Weighted Voting

Some stumps get **more say** in the final classification based on their accuracy.

3. Sequential Learning

Each stump is built by taking the **previous stumps' mistakes** into account.

Key Insight

Unlike Random Forest, AdaBoost uses stumps (not full trees), assigns different weights to different stumps, and builds stumps **sequentially** rather than independently.

Decision Stumps: The Weak Learners

Random Forest vs. AdaBoost

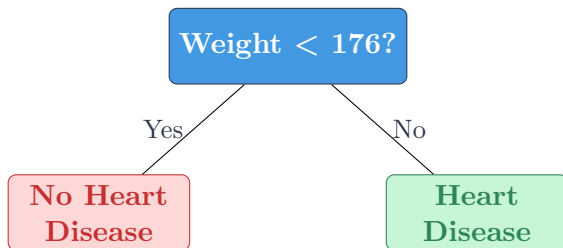
Random Forest

- Uses **full-sized** decision trees
- Each tree uses all variables
- Trees are **independent**
- **Equal** voting power

AdaBoost

- Uses **decision stumps**
- Each stump uses **only one** variable
- Stumps are **sequential/dependent**
- **Weighted** voting power

What is a Decision Stump?



A decision stump has **one internal node** and **two leaf nodes**. It splits on a single feature.

Important

Stumps are **weak learners** — they are not great at making accurate classifications on their own, but AdaBoost combines many of them to create a **strong classifier**.

Example Dataset

Heart Disease Prediction Dataset

Sample	Chest Pain	Blocked Arteries	Weight	Heart Disease	Init. Sample Wt.
1	Yes	Yes	205	Yes	1/8
2	No	No	180	Yes	1/8
3	Yes	No	210	Yes	1/8
4	Yes	Yes	167	Yes	1/8
5	No	No	156	No	1/8
6	No	Yes	125	No	1/8
7	No	No	175	No	1/8
8	Yes	No	177	Yes	1/8

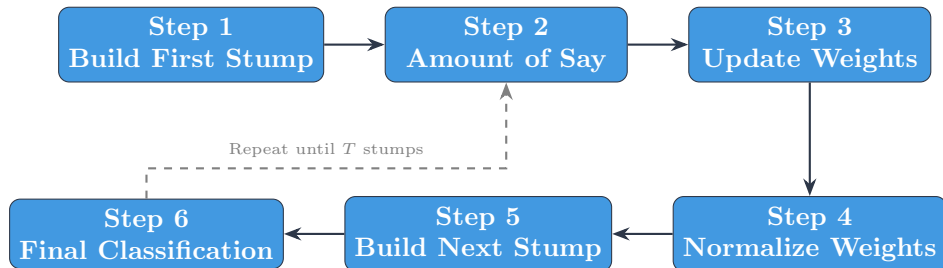
Initial Sample Weights

At the start, all samples receive the same sample weight:

$$w_i^{(1)} = \frac{1}{N} = \frac{1}{8}$$

AdaBoost Step-by-Step

Step-by-Step Roadmap



Step 1: Building the First Stump — Weighted Gini Index

To choose the best stump, we compute the **Weighted Gini Index** for every possible split:

$$\text{Weighted Gini} = \sum_j \frac{N_j}{N} \left(1 - \sum_k \left(\frac{\sum_{i \in S_j, y_i=k} w_i}{\sum_{i \in S_j} w_i} \right)^2 \right)$$

where N_j = samples in node j , S_j = sample set of node j , w_i = weight of sample i , and k = class label.

Goal

The split with the **lowest** Weighted Gini Index becomes the stump.

Step 1: Gini Calculation — Patient Weight < 176

Left Node (Weight < 176):

Samples: 4, 5, 6, 7

- Heart Disease: Sample 4 ($w = 1/8$)
- No HD: Samples 5, 6, 7 ($w = 3/8$)
- $p_{\text{HD}} = \frac{1/8}{4/8} = 0.25$
- $p_{\overline{\text{HD}}} = 0.75$

$$G_{\text{left}} = 1 - (0.25^2 + 0.75^2) = 0.375$$

Right Node (Weight ≥ 176):

Samples: 1, 2, 3, 8

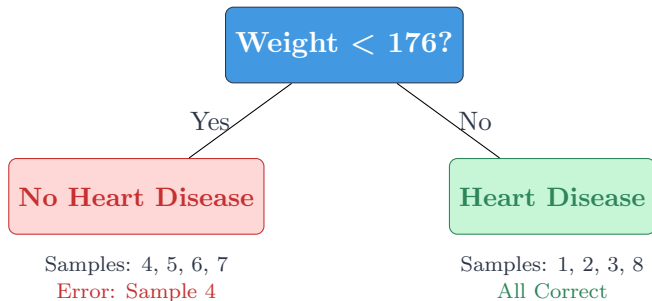
- All have Heart Disease
- $p_{\text{HD}} = 1.0$

$$G_{\text{right}} = 1 - (1.0^2 + 0.0^2) = 0$$

Weighted Average

$$\text{Weighted Gini} = \frac{4}{8} \times 0.375 + \frac{4}{8} \times 0 = \boxed{0.1875}$$

Step 1: First Stump — Patient Weight < 176



Performance: 1 error out of 8 samples (Sample 4 is misclassified).
This stump has the **lowest Weighted Gini** among all candidate splits.

Note: Chest Pain also achieves $Gini = 0.1875$ (a tie). Weight is chosen by convention.

Step 2: Amount of Say (Influence)

Each stump m receives a value α_m called the **Amount of Say** (or **Influence**). It determines **how much voting power** stump m gets in the final classification.

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_m}{\varepsilon_m} \right)$$

where ε_m = weighted error = sum of weights of misclassified samples.

Low error $\Rightarrow$ large α_m (more say), while high error $\Rightarrow$ small or negative α_m (less say).

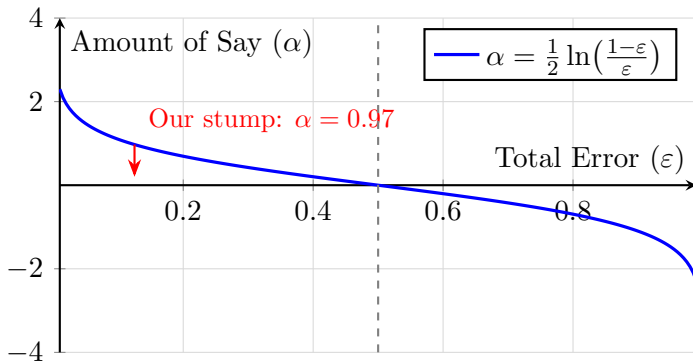
First Stump Calculation

Total error: $\varepsilon_1 = 1/8 = 0.125$ (only Sample 4 is wrong)

$$\alpha_1 = \frac{1}{2} \ln \left(\frac{1 - 1/8}{1/8} \right) = \frac{1}{2} \ln(7) = 0.973$$

This stump has a **high amount of say** ($\alpha_1 = 0.973$) because it makes very few errors.

Step 2: Amount of Say vs. Total Error



Small error $\Rightarrow$ large positive α ; Error = 0.5 $\Rightarrow \alpha = 0$; Large error $\Rightarrow$ large negative α (flips prediction).

Usually we consider stumps where $\alpha > 0$.

Step 3: Update Sample Weights

$$w_i^{(\text{new})} = w_i^{(\text{old})} \cdot e^{-\alpha_m \cdot y_i \cdot h_m(x_i)}$$

Label encoding: $y_i, h_m(x_i) \in \{+1, -1\}$ where $+1$ = Heart Disease and -1 = No Heart Disease.

Assuming $\alpha_m > 0$:

Misclassified: $y_i \neq h_m \Rightarrow y_i \cdot h_m = -1$

$$w_i^{(\text{new})} = w_i^{(\text{old})} \cdot e^{+\alpha_m}$$

Weight **increases** $\uparrow$

Correct: $y_i = h_m \Rightarrow y_i \cdot h_m = +1$

$$w_i^{(\text{new})} = w_i^{(\text{old})} \cdot e^{-\alpha_m}$$

Weight **decreases** $\downarrow$

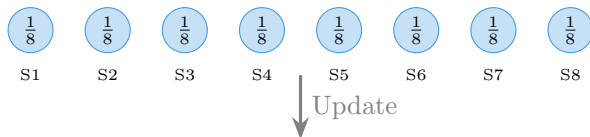
Numerical Example ($\alpha_1 = 0.973$)

Misclassified (Sample 4): $w_4 = \frac{1}{8} \cdot e^{+0.973} = \frac{1}{8} \cdot 2.646 = 0.33$

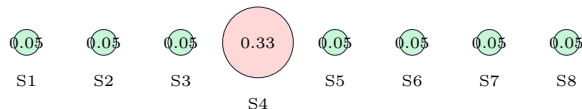
Correctly classified: $w_i = \frac{1}{8} \cdot e^{-0.973} = \frac{1}{8} \cdot 0.378 = 0.05 \ (i \neq 4)$

Step 3: Intuition — Focus on Mistakes

Before (equal sample weights)



After (Sample 4 emphasized)



Key Idea

The misclassified sample gets a **much larger sample weight**, forcing the next stump to focus on getting it right.

Step 4: Normalize the New Sample Weights

$$w_i^{(\text{norm})} = \frac{w_i^{(\text{new})}}{\sum_{j=1}^N w_j^{(\text{new})}}$$

This ensures all sample weights sum to 1.

Sample	Old Sample Weight	New Sample Weight	Normalized
1	0.125	0.05	0.07
2	0.125	0.05	0.07
3	0.125	0.05	0.07
4	0.125	0.33	0.50
5	0.125	0.05	0.07
6	0.125	0.05	0.07
7	0.125	0.05	0.07
8	0.125	0.05	0.07

Step 5: Building the Second Stump — Two Methods

Method 1: Weighted Gini Index

Use the new sample weights **directly** in the Gini calculation to find the best split.

$$\text{Gini}_j = 1 - \sum_k \left(\frac{\sum_{i \in S_j, y_i = k} w_i}{\sum_{i \in S_j} w_i} \right)^2$$

Method 2: Resampling

Create a **new dataset** by sampling with replacement using the normalized sample weights as probabilities.

Higher-weight samples are picked more often.

Step 5: Resampling with Cumulative Distribution

Sample	Sample Weight	Cumulative	Probability Range
1	0.07	0.07	[0.00, 0.07)
2	0.07	0.14	[0.07, 0.14)
3	0.07	0.21	[0.14, 0.21)
4	0.50	0.71	[0.21, 0.71)
5	0.07	0.78	[0.71, 0.78)
6	0.07	0.85	[0.78, 0.85)
7	0.07	0.92	[0.85, 0.92)
8	0.07	1.00	[0.92, 1.00]

Example Random Draws

0.42 $\rightarrow$ S4, 0.05 $\rightarrow$ S1, 0.63 $\rightarrow$ S4, 0.33 $\rightarrow$ S4, 0.55 $\rightarrow$ S4, 0.12 $\rightarrow$ S2, 0.89 $\rightarrow$ S7,
0.95 $\rightarrow$ S8

Resampled: {S4, S1, S4, S4, S4, S2, S7, S8} — Sample 4 appears **4 out of 8** times (= 50%).

Step 5: The Resampled Dataset

After resampling, each sample in the new dataset gets **equal sample weight** = $1/8$.

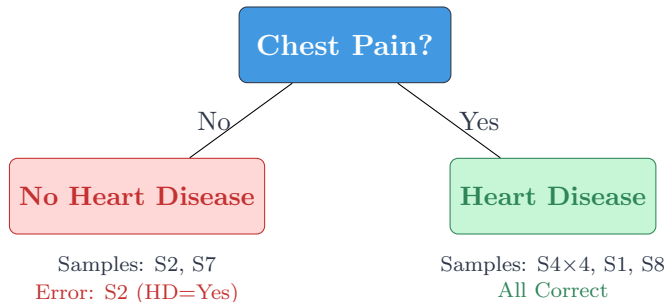
#	Source	Chest Pain	Blocked Art.	Weight	Heart Disease
1	S4	Yes	Yes	167	Yes
2	S1	Yes	Yes	205	Yes
3	S4	Yes	Yes	167	Yes
4	S4	Yes	Yes	167	Yes
5	S4	Yes	Yes	167	Yes
6	S2	No	No	180	Yes
7	S7	No	No	175	No
8	S8	Yes	No	177	Yes

Key Observation

Sample 4 (the one misclassified by the first stump) now appears **4 times** in the new dataset. The second stump is forced to give more importance to correctly classifying this sample — this is how AdaBoost **focuses on mistakes!**

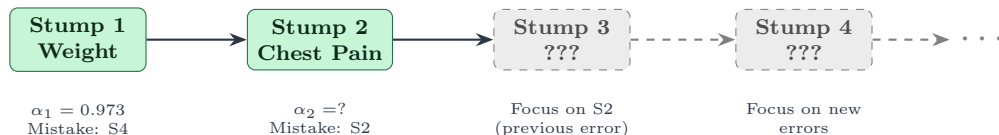
Step 5: Second Stump — Chest Pain

We build a stump on the **resampled dataset**. Since Sample 4 (Chest Pain = Yes, HD = Yes) appears 4 times, the stump must classify it correctly.



Chest Pain splits the resampled data well: all Chest Pain = Yes samples have HD = Yes. Only S2 (CP = No, HD = Yes) is misclassified. The Amount of Say α_2 is computed from this error.

Continuing the Process...



The Pattern

After each stump, we repeat the same process:

- 1 Compute α (Amount of Say) for the current stump
- 2 Update and normalize sample weights (emphasizing new mistakes)
- 3 Build the next stump on the re-weighted/resampled data

Each new stump **corrects the mistakes** of all previous stumps. We continue until we reach T stumps or training error is zero.

Step 6: The Strong Classifier

$$H(x) = \text{sign} \left(\sum_{m=1}^M \alpha_m h_m(x) \right)$$

where α_m = amount of say of stump m , $h_m(x) \in \{+1, -1\}$ = prediction of stump m .

Hypothetical Example: Classifying a New Patient

The α values below are illustrative and not derived from our running example.

Stump	Prediction	α	Weighted Vote
1 (Weight)	+1 (Heart Disease)	0.973	+0.973
2 (Chest Pain)	-1 (No HD)	0.421	-0.421
3 (Arteries)	+1 (Heart Disease)	0.651	+0.651
Sum:			+1.203

$\text{sign}(+1.203) = +1 \implies$ **Heart Disease**

Step 6: Ensemble of Stumps



Each stump contributes to the final decision with its **weighted vote**.
Better stumps (lower error) get **more influence**.

Algorithm Summary

Complete AdaBoost Algorithm

Input: Training set $\{(x_1, y_1), \dots, (x_N, y_N)\}$, number of rounds T .

❶ **Initialize:** $w_i^{(1)} = \frac{1}{N}$ for $i = 1, \dots, N$

❷ **For** $t = 1$ **to** T :

(a) Train weak learner h_t using weights $w^{(t)}$

(b) Weighted error: $\varepsilon_t = \sum_{i: h_t(x_i) \neq y_i} w_i^{(t)}$

(c) Influence: $\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right)$

(d) Update: $w_i^{(t+1)} = w_i^{(t)} \cdot e^{-\alpha_t y_i h_t(x_i)}$

(e) Normalize: $w_i^{(t+1)} \leftarrow \frac{w_i^{(t+1)}}{\sum_j w_j^{(t+1)}}$

❸ **Output:** $H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$

Key Equations at a Glance

Initial weights: $w_i^{(1)} = \frac{1}{N}$ (1)

Weighted error: $\varepsilon_t = \sum_{i=1}^N w_i^{(t)} \mathbf{1}[h_t(x_i) \neq y_i]$ (2)

Amount of Say: $\alpha_t = \frac{1}{2} \ln\left(\frac{1 - \varepsilon_t}{\varepsilon_t}\right)$ (3)

Weight update: $w_i^{(t+1)} = w_i^{(t)} \exp(-\alpha_t y_i h_t(x_i))$ (4)

Normalization: $w_i^{(t+1)} \leftarrow \frac{w_i^{(t+1)}}{\sum_{j=1}^N w_j^{(t+1)}}$ (5)

Final classifier: $H(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$ (6)

Why Does AdaBoost Work?

Three Reasons AdaBoost Works

Focus on Mistakes

Each new stump focuses on samples that **previous stumps got wrong**, gradually improving overall accuracy.

Weighted Voting

Better stumps (lower error) get **more influence**, ensuring quality predictions dominate.

Margin Maximization

AdaBoost maximizes the **margin** of predictions, leading to better generalization.

Margin Definition

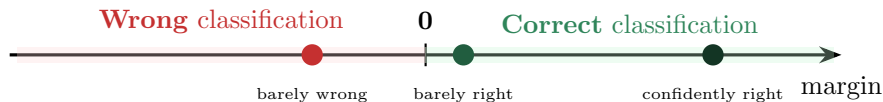
$$\text{margin}(x, y) = y \cdot \sum_{t=1}^T \alpha_t h_t(x)$$

Larger margins $\Rightarrow$ more confident and reliable predictions.

How Margin Maximization Works

For a sample (x_i, y_i) , the **margin** measures how *confidently* AdaBoost classifies it:

$$\text{margin}(x_i, y_i) = y_i \cdot \underbrace{\sum_{t=1}^T \alpha_t h_t(x_i)}_{\text{weighted vote}}$$

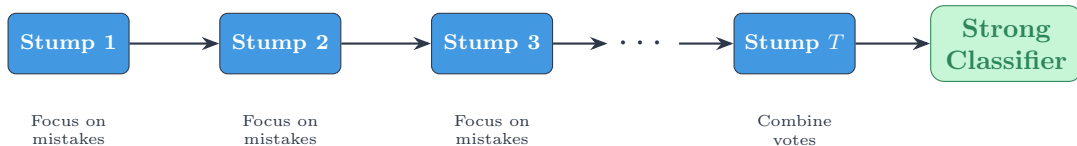


- Margin > 0 : correctly classified
- Margin < 0 : misclassified
- **Larger margin** = more confidence

Why it matters

By increasing weights on low-margin samples, AdaBoost pushes their margins up, making the **overall classifier more robust** to unseen data.

AdaBoost Process Flow



Key Insight

Even when training error reaches zero, AdaBoost can continue running to **increase margins** and improve generalization to new data.

Practical Considerations

When to Stop & Edge Cases

Stopping Criteria

- **Fixed rounds:** Set T in advance
- **Validation-based:** Stop when validation error plateaus
- **Perfect training:** Stop when training error = 0

Edge Case: $\varepsilon = 0$ or $\varepsilon = 1$

Add a small $\delta > 0$ to prevent division by zero:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t + \delta}{\varepsilon_t + \delta} \right)$$

Requirement

Weak learners should have error < 0.5 (better than random). If error > 0.5 , the negative α **flips the prediction**, which can still be useful! However, we usually don't consider the stumps where error > 0.5 !

Applications & Limitations

Applications & Advantages

Common Applications

- **Face Detection:** Viola–Jones algorithm
- **Text Classification:** Spam filtering
- **Medical Diagnosis:** Binary classification
- **Object Recognition:** Computer vision

Advantages

- Simple to understand and implement
- Flexible: works with any weak learner
- Minimal hyperparameter tuning
- Often very effective in practice

Limitations

Sensitive to noise and outliers (they get high weights); can overfit on noisy data; training is sequential and **cannot be parallelized**; requires weak learners better than random chance.

Final Review

The Three Ideas — Revisited

1. Combine Weak Learners

Many **stumps** combined into one classifier.

$$\sum_{t=1}^T \alpha_t h_t(x)$$

2. Weighted Voting

Each stump's **influence** is based on its accuracy.

$$\alpha = \frac{1}{2} \ln\left(\frac{1-\varepsilon}{\varepsilon}\right)$$

3. Learn from Mistakes

Weights shift focus to **previous errors**.

$$w_i \leftarrow w_i \cdot e^{-\alpha y_i h(x_i)}$$

AdaBoost = **Ad**aptive **Boo**sting

“Boost the performance of weak learners by adapting to their mistakes!”

Thank You!

Questions?